

Scala Hands-On Assessment Deck

1. [Basic] – Standard Functions & Return Types

Scenario: E-commerce Order Pricing Service.

Requirements:

- Define function `calculateFinalAmount(basePrice: Double, taxPercentage: Double, discountAmount: Double): Double`
- Add tax and subtract discount.
- Print final amount.

Constraints:

- Explicit return types.
- No `var`.
- No `null`.
- Prefer immutability.

Grading Checklist:

- Explicit return type
- No mutable variables
- Clean arithmetic logic
- Proper naming conventions

2. [Basic] – Varargs (Variable-Length Arguments)

Scenario: Shopping Cart Service.

Requirements:

- Function `calculateCartTotal(prices: Double*)`: Double
- Print number of items.
- Demonstrate calls with 1, 3, 5 items.

Constraints:

- Must use varargs syntax.
- No loops.
- Immutable style.

Grading Checklist:

- Correct varargs syntax
- Functional aggregation
- Immutable logic

3. [Basic] – Lambda Expressions & Syntax

Scenario: User Management System.

Requirements:

- Create list of users and active flags.
- Use lambda to filter active users.
- Print active users.

Constraints:

- Must use lambda syntax.
- No loops.
- Immutable collections.

Grading Checklist:

- Proper lambda syntax
- Uses filter/map
- Functional style

4. [Intermediate] – Pure vs Impure Functions

Scenario: Banking Transaction Module.

Requirements:

- Create one pure function (calculate balance).
- Create one impure function (logging).
- Demonstrate both.

Constraints:

- Pure function has no side effects.
- No global mutable state.

Grading Checklist:

- Clear separation of concerns
- Deterministic pure function
- Logging in impure function only

5. [Intermediate] – Higher-Order Functions

Scenario: Payment Processing Engine.

Requirements:

- `processPayment(amount: Double, feeStrategy: Double => Double): Double`
- Create at least two fee strategies.
- Demonstrate calling HOF.

Constraints:

- No mutable variables.
- Must return value.

Grading Checklist:

- Correct function type syntax
- Lambda strategies
- Clean separation of logic

6. [Intermediate] – Classes & Inheritance

Scenario: Loan Management System.

Requirements:

- Parent class Loan with calculateInterest.
- Child classes HomeLoan and CarLoan.
- Override interest logic.
- Demonstrate object usage.

Constraints:

- Proper extends usage.
- Avoid mutable state.

Grading Checklist:

- Proper inheritance
- Correct override usage
- Clean class design

7. [Advanced] – Method Overloading vs Overriding

Scenario: Notification Service.

Requirements:

- Base class NotificationService.
- Overload sendNotification.
- Child class EmailNotificationService overrides method.
- Demonstrate polymorphism.

Constraints:

- Use override keyword.
- No mutable state.

Grading Checklist:

- Correct overloaded signatures
- Proper overriding
- Demonstrates runtime polymorphism

8. [Advanced] – Abstract Classes, Traits & Multiple Trait Mixins

Scenario: Microservices Logging Framework.

Requirements:

- Abstract class Service with execute().
- Traits Logging and Auditing.
- PaymentService extends Service and mixes traits.
- Demonstrate usage.

Constraints:

- Implement abstract method.
- Mix multiple traits.
- No null usage.

Grading Checklist:

- Proper abstract class
- Multiple trait mixins
- Clean hybrid OOP + FP design